Squirrel

Generated by Doxygen 1.9.8

Chapter 1

File Index

1.1 File List

Here is a list of all files with brief descriptions:

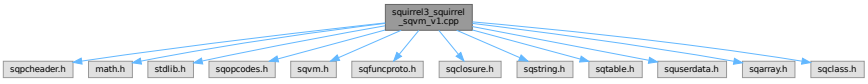
Chapter 2

File Documentation

2.1 squirrel3_squirrel_sqvm_v1.cpp File Reference

```
#include "sqpcheader.h"
#include <math.h>
#include <stdlib.h>
#include "sqopcodes.h"
#include "sqvm.h"
#include "sqfuncproto.h"
#include "sqclosure.h"
#include "sqstring.h"
#include "sqtable.h"
#include "squserdata.h"
#include "sqarray.h"
#include "sqclass.h"
```

Include dependency graph for squirrel3_squirrel_sqvm_v1.cpp:



Macros

- #define TOP() (_stack._vals[_top-1])

  スタックのトップにある値を取得します。
- #define TARGET _stack._vals[_stackbase+arg0]

  命令のターゲットレジスタ（スタック上の特定の場所）を取得します。
- #define STK(a) _stack._vals[_stackbase+(a)]

  現在のスタックベースからの相対位置でスタック上の値を取得します。
- #define _ARITH_(op, trg, o1, o2)

  算術演算を実行するためのマクロ。
- #define _ARITH_NOZERO(op, trg, o1, o2, err)

  ゼロ除算チェック付きの算術演算を実行するためのマクロ。
- #define _RET_SUCCEED(exp) { result = (exp); return true; }

  比較結果を設定して *true* を返すマクロ。

- #define _RET_ON_FAIL(exp) { if(!exp) return false; }
  式の評価が失敗した場合に *false* を返すマクロ。
- #define arg0 (_i_._arg0)
  現在の命令の *0* 番目の引数。
- #define sarg0 ((SQInteger)*((const signed char *)&_i_._arg0))
  現在の命令の *0* 番目の引数（符号付き）。
- #define arg1 (_i_._arg1)
  現在の命令の *1* 番目の引数。
- #define sarg1 (*((const SQInt32 *)&_i_._arg1))
  現在の命令の *1* 番目の引数（符号付き）。
- #define arg2 (_i_._arg2)
  現在の命令の *2* 番目の引数。
- #define arg3 (_i_._arg3)
  現在の命令の *3* 番目の引数。
- #define sarg3 ((SQInteger)*((const signed char *)&_i_._arg3))
  現在の命令の *3* 番目の引数（符号付き）。
- #define _FINISH(howmuchtojump) {jump = howmuchtojump; return true; }
  ***foreach* ループの反復処理を終了し、ジャンプオフセットを設定するマクロ。**
- #define COND_LITERAL (arg3!=0?ci->_literals[arg1]:STK(arg1))
  条件付きでリテラルまたはスタック上の値を取得するマクロ。
- #define SQ_THROW() { goto exception_trap; }
  例外トラップへジャンプするマクロ。
- #define _GUARD(exp) { if(!exp) { SQ_THROW();} }
  式を評価し、失敗した場合に例外をスローするマクロ。
- #define FALLBACK_OK 0
  フォールバック*Get/Set* 操作が成功したことを示す定数。
- #define FALLBACK_NO_MATCH 1
  フォールバック*Get/Set* 操作で一致するものがなかったことを示す定数。
- #define FALLBACK_ERROR 2
  フォールバック*Get/Set* 操作中にエラーが発生したことを示す定数。

Variables

- SQInstructionDesc g_InstrDesc [ ]
  命令セットのディスクリプション配列（外部リンケージ）。デバッグ用。

2.1.1 Macro Definition Documentation

2.1.1.1 _ARITH_

```
#define _ARITH_(
                op,
                trg,
                o1,
                o2 )
```

Value:

```
{ \
    SQInteger tmask = sq_type(o1)|sq_type(o2); \
    switch(tmask) { \
        case OT_INTEGER: trg = _integer(o1) op _integer(o2);break; \
        case (OT_FLOAT|OT_INTEGER): \
```

```
        case (OT_FLOAT): trg = tofloat(o1) op tofloat(o2); break;\
        default: _GUARD(ARITH_OP((#op)[0],trg,o1,o2)); break;\
    } \
}
```

算術演算を実行するためのマクロ。

オブジェクトの型に応じて整数または浮動小数点数の演算を高速に実行します。 それ以外の型の場合は ARITH_OP 関数にフォールバックします。

Parameters

op	算術演算子。
trg	結果を格納するSQObjectPtr。
o1	最初のオペランド。
o2	2 番目のオペランド。

Definition at line 73 of file squirrel3_squirrel_sqvm_v1.cpp.

```
00074 { \
00075     SQInteger tmask = sq_type(o1)|sq_type(o2); \
00076     switch(tmask) { \
00077         case OT_INTEGER: trg = _integer(o1) op _integer(o2);break; \
00078         case (OT_FLOAT|OT_INTEGER): \
00079         case (OT_FLOAT): trg = tofloat(o1) op tofloat(o2); break;\
00080         default: _GUARD(ARITH_OP((#op)[0],trg,o1,o2)); break;\
00081     } \
00082 }
```

2.1.1.2 _ARITH_NOZERO

```
#define _ARITH_NOZERO(
                op,
                trg,
                o1,
                o2,
                err )
```

Value:

```
{ \
    SQInteger tmask = sq_type(o1)|sq_type(o2); \
    switch(tmask) { \
        case OT_INTEGER: { SQInteger i2 = _integer(o2); if(i2 == 0) { Raise_Error(err); SQ_THROW(); } trg =
      _integer(o1) op i2; } break;\
        case (OT_FLOAT|OT_INTEGER): \
        case (OT_FLOAT): trg = tofloat(o1) op tofloat(o2); break;\
        default: _GUARD(ARITH_OP((#op)[0],trg,o1,o2)); break;\
    } \
}
```

ゼロ除算チェック付きの算術演算を実行するためのマクロ。

_ARITH_マクロに似ていますが、整数除算の際にゼロ除算をチェックし、エラーを発生させます。

Parameters

op	算術演算子。
trg	結果を格納するSQObjectPtr。
o1	最初のオペランド。
o2	2 番目のオペランド。
err	ゼロ除算の場合のエラーメッセージ。

Definition at line 93 of file squirrel3_squirrel_sqvm_v1.cpp.

```
00094 { \
00095     SQInteger tmask = sq_type(o1)|sq_type(o2); \
00096     switch(tmask) { \
00097         case OT_INTEGER: { SQInteger i2 = _integer(o2); if(i2 == 0) { Raise_Error(err); SQ_THROW(); }
      trg = _integer(o1) op i2; } break;\
00098         case (OT_FLOAT|OT_INTEGER): \
00099         case (OT_FLOAT): trg = tofloat(o1) op tofloat(o2); break;\
00100         default: _GUARD(ARITH_OP((#op)[0],trg,o1,o2)); break;\
00101     } \
00102 }
```

2.1.1.3 _FINISH

```
#define _FINISH(
                howmuchtojump ) {jump = howmuchtojump; return true; }
```

foreach ループの反復処理を終了し、ジャンプオフセットを設定するマクロ。

Parameters

howmuchtojump	IP を進めるオフセット値。

Definition at line 721 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.4 _GUARD

```
#define _GUARD(
                exp ) { if(!exp) { SQ_THROW();} }
```

式を評価し、失敗した場合に例外をスローするマクロ。

Definition at line 802 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.5 _RET_ON_FAIL

```
#define _RET_ON_FAIL(
                exp ) { if(!exp) return false; }
```

式の評価が失敗した場合に false を返すマクロ。

Parameters

exp	評価する式。

Definition at line 644 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.6 _RET_SUCCEED

```
#define _RET_SUCCEED(
                exp ) { result = (exp); return true; }
```

比較結果を設定して true を返すマクロ。

ObjCmp 関数内で使用され、コードを簡潔にします。

Parameters

exp	比較結果の式。

Definition at line 291 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.7 arg0

```
#define arg0 (_i_._arg0)
```

現在の命令の 0 番目の引数。

Definition at line 688 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.8 arg1

```
#define arg1 (_i_._arg1)
```

現在の命令の 1 番目の引数。

Definition at line 692 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.9 arg2

```
#define arg2 (_i_._arg2)
```

現在の命令の 2 番目の引数。

Definition at line 696 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.10 arg3

```
#define arg3 (_i_._arg3)
```

現在の命令の 3 番目の引数。

Definition at line 698 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.11 COND_LITERAL

```
#define COND_LITERAL (arg3!=0?ci->_literals[arg1]:STK(arg1))
```

条件付きでリテラルまたはスタック上の値を取得するマクロ。

Definition at line 796 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.12 FALLBACK_ERROR

```
#define FALLBACK_ERROR 2
```

フォールバックGet/Set 操作中にエラーが発生したことを示す定数。

Definition at line 1573 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.13 FALLBACK_NO_MATCH

```
#define FALLBACK_NO_MATCH 1
```

フォールバックGet/Set 操作で一致するものがなかったことを示す定数。

Definition at line 1571 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.14 FALLBACK_OK

```
#define FALLBACK_OK 0
```

フォールバックGet/Set 操作が成功したことを示す定数。

Definition at line 1569 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.15 sarg0

```
#define sarg0 ((SQInteger)*((const signed char *)&_i_._arg0))
```

現在の命令の 0 番目の引数（符号付き）。

Definition at line 690 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.16 sarg1

```
#define sarg1 (*((const SQInt32 *)&_i_._arg1))
```

現在の命令の 1 番目の引数（符号付き）。

Definition at line 694 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.17 sarg3

```
#define sarg3 ((SQInteger)*((const signed char *)&_i_._arg3))
```

現在の命令の 3 番目の引数（符号付き）。

Definition at line 700 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.18 SQ_THROW

```
#define SQ_THROW( ) { goto exception_trap; }
```

例外トラップへジャンプするマクロ。

Definition at line 799 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.19 STK

```
#define STK(
              a ) _stack._vals[_stackbase+(a)]
```

現在のスタックベースからの相対位置でスタック上の値を取得します。

Parameters

a	スタックベースからのオフセット。

Definition at line 31 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.20 TARGET

```
#define TARGET _stack._vals[_stackbase+arg0]
```

命令のターゲットレジスタ（スタック上の特定の場所）を取得します。

Definition at line 26 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.1.21 TOP

```
#define TOP( ) (_stack._vals[_top-1])
```

スタックのトップにある値を取得します。

Definition at line 22 of file squirrel3_squirrel_sqvm_v1.cpp.

2.1.2 Variable Documentation

2.1.2.1 g_InstrDesc

```
SQInstructionDesc g_InstrDesc[ ]  [extern]
```

命令セットのディスクリプション配列（外部リンケージ）。デバッグ用。

2.2 squirrel3_squirrel_sqvm_v1.cpp

Go to the documentation of this file.

```
00001 //cpp
00002
00003 /*
00004     Written By Gemini
00005 */
00006 #include "sqpcheader.h"
00007 #include <math.h>
00008 #include <stdlib.h>
00009 #include "sqopcodes.h"
00010 #include "sqvm.h"
00011 #include "sqfuncproto.h"
00012 #include "sqclosure.h"
00013 #include "sqstring.h"
00014 #include "sqtable.h"
00015 #include "squserdata.h"
00016 #include "sqarray.h"
00017 #include "sqclass.h"
00018
00022 #define TOP() (_stack._vals[_top-1])
00026 #define TARGET _stack._vals[_stackbase+arg0]
00031 #define STK(a) _stack._vals[_stackbase+(a)]
00032
00043 bool SQVM::BW_OP(SQUnsignedInteger op,SQObjectPtr &trg,const SQObjectPtr &o1,const SQObjectPtr &o2)
```

```
00044 {
00045     SQInteger res;
00046     if((sq_type(o1)| sq_type(o2)) == OT_INTEGER)
00047     {
00048         SQInteger i1 = _integer(o1), i2 = _integer(o2);
00049         switch(op) {
00050             case BW_AND:    res = i1 & i2; break;
00051             case BW_OR:     res = i1 | i2; break;
00052             case BW_XOR:    res = i1 ^ i2; break;
00053             case BW_SHIFTL: res = i1 « i2; break;
00054             case BW_SHIFTR: res = i1 » i2; break;
00055             case BW_USHIFTR:res = (SQInteger)(*((SQUnsignedInteger*)&i1) » i2); break;
00056             default: { Raise_Error(_SC("internal vm error bitwise op failed")); return false; }
00057         }
00058     }
00059     else { Raise_Error(_SC("bitwise op between '%s' and '%s'"),GetTypeName(o1),GetTypeName(o2));
      return false;}
00060     trg = res;
00061     return true;
00062 }
00063
00073 #define _ARITH_(op,trg,o1,o2) \
00074 { \
00075     SQInteger tmask = sq_type(o1)|sq_type(o2); \
00076     switch(tmask) { \
00077         case OT_INTEGER: trg = _integer(o1) op _integer(o2);break; \
00078         case (OT_FLOAT|OT_INTEGER): \
00079         case (OT_FLOAT): trg = tofloat(o1) op tofloat(o2); break;\
00080         default: _GUARD(ARITH_OP((#op)[0],trg,o1,o2)); break;\
00081     } \
00082 }
00083
00093 #define _ARITH_NOZERO(op,trg,o1,o2,err) \
00094 { \
00095     SQInteger tmask = sq_type(o1)|sq_type(o2); \
00096     switch(tmask) { \
00097         case OT_INTEGER: { SQInteger i2 = _integer(o2); if(i2 == 0) { Raise_Error(err); SQ_THROW(); }
      trg = _integer(o1) op i2; } break;\
00098         case (OT_FLOAT|OT_INTEGER): \
00099         case (OT_FLOAT): trg = tofloat(o1) op tofloat(o2); break;\
00100         default: _GUARD(ARITH_OP((#op)[0],trg,o1,o2)); break;\
00101     } \
00102 }
00103
00114 bool SQVM::ARITH_OP(SQUnsignedInteger op,SQObjectPtr &trg,const SQObjectPtr &o1,const SQObjectPtr &o2)
00115 {
00116     SQInteger tmask = sq_type(o1)| sq_type(o2);
00117     switch(tmask) {
00118         case OT_INTEGER:{
00119             SQInteger res, i1 = _integer(o1), i2 = _integer(o2);
00120             switch(op) {
00121             case '+': res = i1 + i2; break;
00122             case '-': res = i1 - i2; break;
00123             case '/': if (i2 == 0) { Raise_Error(_SC("division by zero")); return false; }
00124                     else if (i2 == -1 && i1 == INT_MIN) { Raise_Error(_SC("integer overflow")); return
      false; }
00125                     res = i1 / i2;
00126                     break;
00127             case '*': res = i1 * i2; break;
00128             case '%': if (i2 == 0) { Raise_Error(_SC("modulo by zero")); return false; }
00129                     else if (i2 == -1 && i1 == INT_MIN) { res = 0; break; }
00130                     res = i1 % i2;
00131                     break;
00132             default: res = 0xDEADBEEF;
00133             }
00134             trg = res; }
00135             break;
00136         case (OT_FLOAT|OT_INTEGER):
00137         case (OT_FLOAT):{
00138             SQFloat res, f1 = tofloat(o1), f2 = tofloat(o2);
00139             switch(op) {
00140             case '+': res = f1 + f2; break;
00141             case '-': res = f1 - f2; break;
00142             case '/': res = f1 / f2; break;
00143             case '*': res = f1 * f2; break;
00144             case '%': res = SQFloat(fmod((double)f1,(double)f2)); break;
00145             default: res = 0x0f;
00146             }
00147             trg = res; }
00148             break;
00149         default:
00150             if(op == '+' && (tmask & _RT_STRING)){
00151                 if(!StringCat(o1, o2, trg)) return false;
00152             }
00153             else if(!ArithMetaMethod(op,o1,o2,trg)) {
00154                 return false;
00155             }
```

```
00156     }
00157     return true;
00158 }
00159
00165 SQVM::SQVM(SQSharedState *ss)
00166 {
00167     _sharedstate=ss;
00168     _suspended = SQFalse;
00169     _suspended_target = -1;
00170     _suspended_root = SQFalse;
00171     _suspended_traps = -1;
00172     _foreignptr = NULL;
00173     _nnativecalls = 0;
00174     _nmetamethodscall = 0;
00175     _lasterror.Null();
00176     _errorhandler.Null();
00177     _debughook = false;
00178     _debughook_native = NULL;
00179     _debughook_closure.Null();
00180     _openouters = NULL;
00181     ci = NULL;
00182     _releasehook = NULL;
00183     INIT_CHAIN();ADD_TO_CHAIN(&_ss(this)->_gc_chain,this);
00184 }
00185
00190 void SQVM::Finalize()
00191 {
00192     if(_releasehook) { _releasehook(_foreignptr,0); _releasehook = NULL; }
00193     if(_openouters) CloseOuters(&_stack._vals[0]);
00194     _roottable.Null();
00195     _lasterror.Null();
00196     _errorhandler.Null();
00197     _debughook = false;
00198     _debughook_native = NULL;
00199     _debughook_closure.Null();
00200     temp_reg.Null();
00201     _callstackdata.resize(0);
00202     SQInteger size=_stack.size();
00203     for(SQInteger i=0;i<size;i++)
00204         _stack[i].Null();
00205 }
00206
00211 SQVM::~SQVM()
00212 {
00213     Finalize();
00214     REMOVE_FROM_CHAIN(&_ss(this)->_gc_chain,this);
00215 }
00216
00226 bool SQVM::ArithMetaMethod(SQInteger op,const SQObjectPtr &o1,const SQObjectPtr &o2,SQObjectPtr &dest)
00227 {
00228     SQMetaMethod mm;
00229     switch(op){
00230         case _SC('+'): mm=MT_ADD; break;
00231         case _SC('-'): mm=MT_SUB; break;
00232         case _SC('/'): mm=MT_DIV; break;
00233         case _SC('*'): mm=MT_MUL; break;
00234         case _SC('%'): mm=MT_MODULO; break;
00235         default: mm = MT_ADD; assert(0); break; //shutup compiler
00236     }
00237     if(is_delegable(o1) && _delegable(o1)->_delegate) {
00238
00239         SQObjectPtr closure;
00240         if(_delegable(o1)->GetMetaMethod(this, mm, closure)) {
00241             Push(o1);Push(o2);
00242             return CallMetaMethod(closure,mm,2,dest);
00243         }
00244     }
00245     Raise_Error(_SC("arith op %c on between '%s' and '%s'"),op,GetTypeName(o1),GetTypeName(o2));
00246     return false;
00247 }
00248
00257 bool SQVM::NEG_OP(SQObjectPtr &trg,const SQObjectPtr &o)
00258 {
00259
00260     switch(sq_type(o)) {
00261     case OT_INTEGER:
00262         trg = -_integer(o);
00263         return true;
00264     case OT_FLOAT:
00265         trg = -_float(o);
00266         return true;
00267     case OT_TABLE:
00268     case OT_USERDATA:
00269     case OT_INSTANCE:
00270         if(_delegable(o)->_delegate) {
00271             SQObjectPtr closure;
00272             if(_delegable(o)->GetMetaMethod(this, MT_UNM, closure)) {
```

```
00273                 Push(o);
00274                 if(!CallMetaMethod(closure, MT_UNM, 1, temp_reg)) return false;
00275                 _Swap(trg,temp_reg);
00276                 return true;
00277
00278             }
00279         }
00280     default:break; //shutup compiler
00281     }
00282     Raise_Error(_SC("attempt to negate a %s"), GetTypeName(o));
00283     return false;
00284 }
00285
00291 #define _RET_SUCCEED(exp) { result = (exp); return true; }
00302 bool SQVM::ObjCmp(const SQObjectPtr &o1,const SQObjectPtr &o2,SQInteger &result)
00303 {
00304     SQObjectType t1 = sq_type(o1), t2 = sq_type(o2);
00305     if(t1 == t2) {
00306         if(_rawval(o1) == _rawval(o2))_RET_SUCCEED(0);
00307         SQObjectPtr res;
00308         switch(t1){
00309         case OT_STRING:
00310             _RET_SUCCEED(scstrcmp(_stringval(o1),_stringval(o2)));
00311         case OT_INTEGER:
00312             _RET_SUCCEED((_integer(o1)<_integer(o2))?-1:1);
00313         case OT_FLOAT:
00314             _RET_SUCCEED((_float(o1)<_float(o2))?-1:1);
00315         case OT_TABLE:
00316         case OT_USERDATA:
00317         case OT_INSTANCE:
00318             if(_delegable(o1)->_delegate) {
00319                 SQObjectPtr closure;
00320                 if(_delegable(o1)->GetMetaMethod(this, MT_CMP, closure)) {
00321                     Push(o1);Push(o2);
00322                     if(CallMetaMethod(closure,MT_CMP,2,res)) {
00323                         if(sq_type(res) != OT_INTEGER) {
00324                             Raise_Error(_SC("_cmp must return an integer"));
00325                             return false;
00326                         }
00327                         _RET_SUCCEED(_integer(res))
00328                     }
00329                     return false;
00330                 }
00331             }
00332             //continues through (no break needed)
00333         default:
00334             _RET_SUCCEED( _userpointer(o1) < _userpointer(o2)?-1:1 );
00335         }
00336         assert(0);
00337         //if(type(res)!=OT_INTEGER) { Raise_CompareError(o1,o2); return false; }
00338         //  _RET_SUCCEED(_integer(res));
00339
00340     }
00341     else{
00342         if(sq_isnumeric(o1) && sq_isnumeric(o2)){
00343             if((t1==OT_INTEGER) && (t2==OT_FLOAT)) {
00344                 if( _integer(o1)==_float(o2) ) { _RET_SUCCEED(0); }
00345                 else if( _integer(o1)<_float(o2) ) { _RET_SUCCEED(-1); }
00346                 _RET_SUCCEED(1);
00347             }
00348             else{
00349                 if( _float(o1)==_integer(o2) ) { _RET_SUCCEED(0); }
00350                 else if( _float(o1)<_integer(o2) ) { _RET_SUCCEED(-1); }
00351                 _RET_SUCCEED(1);
00352             }
00353         }
00354         else if(t1==OT_NULL) {_RET_SUCCEED(-1);}
00355         else if(t2==OT_NULL) {_RET_SUCCEED(1);}
00356         else { Raise_CompareError(o1,o2); return false; }
00357
00358     }
00359     assert(0);
00360     _RET_SUCCEED(0); //cannot happen
00361 }
00362
00372 bool SQVM::CMP_OP(CmpOP op, const SQObjectPtr &o1,const SQObjectPtr &o2,SQObjectPtr &res)
00373 {
00374     SQInteger r;
00375     if(ObjCmp(o1,o2,r)) {
00376         switch(op) {
00377             case CMP_G: res = (r > 0); return true;
00378             case CMP_GE: res = (r >= 0); return true;
00379             case CMP_L: res = (r < 0); return true;
00380             case CMP_LE: res = (r <= 0); return true;
00381             case CMP_3W: res = r; return true;
00382         }
00383         assert(0);
```

```
00384     }
00385     return false;
00386 }
00387
00397 bool SQVM::ToString(const SQObjectPtr &o,SQObjectPtr &res)
00398 {
00399     switch(sq_type(o)) {
00400     case OT_STRING:
00401         res = o;
00402         return true;
00403     case OT_FLOAT:
00404         scsprintf(_sp(sq_rsl(NUMBER_MAX_CHAR+1)),sq_rsl(NUMBER_MAX_CHAR),_SC("%g"),_float(o));
00405         break;
00406     case OT_INTEGER:
00407         scsprintf(_sp(sq_rsl(NUMBER_MAX_CHAR+1)),sq_rsl(NUMBER_MAX_CHAR),_PRINT_INT_FMT,_integer(o));
00408         break;
00409     case OT_BOOL:
00410         scsprintf(_sp(sq_rsl(6)),sq_rsl(6),_integer(o)?_SC("true"):_SC("false"));
00411         break;
00412     case OT_NULL:
00413         scsprintf(_sp(sq_rsl(5)),sq_rsl(5),_SC("null"));
00414         break;
00415     case OT_TABLE:
00416     case OT_USERDATA:
00417     case OT_INSTANCE:
00418         if(_delegable(o)->_delegate) {
00419             SQObjectPtr closure;
00420             if(_delegable(o)->GetMetaMethod(this, MT_TOSTRING, closure)) {
00421                 Push(o);
00422                 if(CallMetaMethod(closure,MT_TOSTRING,1,res)) {
00423                     if(sq_type(res) == OT_STRING)
00424                         return true;
00425                 }
00426                 else {
00427                     return false;
00428                 }
00429             }
00430         }
00431     default:
00432
      scsprintf(_sp(sq_rsl((sizeof(void*)*2)+NUMBER_MAX_CHAR)),sq_rsl((sizeof(void*)*2)+NUMBER_MAX_CHAR),_SC("(%s
      : 0x%p)"),GetTypeName(o),(void*)_rawval(o));
00433     }
00434     res = SQString::Create(_ss(this),_spval);
00435     return true;
00436 }
00437
00438
00447 bool SQVM::StringCat(const SQObjectPtr &str,const SQObjectPtr &obj,SQObjectPtr &dest)
00448 {
00449     SQObjectPtr a, b;
00450     if(!ToString(str, a)) return false;
00451     if(!ToString(obj, b)) return false;
00452     SQInteger l = _string(a)->_len , ol = _string(b)->_len;
00453     SQChar *s = _sp(sq_rsl(l + ol + 1));
00454     memcpy(s, _stringval(a), sq_rsl(l));
00455     memcpy(s + l, _stringval(b), sq_rsl(ol));
00456     dest = SQString::Create(_ss(this), _spval, l + ol);
00457     return true;
00458 }
00459
00468 bool SQVM::TypeOf(const SQObjectPtr &obj1,SQObjectPtr &dest)
00469 {
00470     if(is_delegable(obj1) && _delegable(obj1)->_delegate) {
00471         SQObjectPtr closure;
00472         if(_delegable(obj1)->GetMetaMethod(this, MT_TYPEOF, closure)) {
00473             Push(obj1);
00474             return CallMetaMethod(closure,MT_TYPEOF,1,dest);
00475         }
00476     }
00477     dest = SQString::Create(_ss(this),GetTypeName(obj1));
00478     return true;
00479 }
00480
00489 bool SQVM::Init(SQVM *friendvm, SQInteger stacksize)
00490 {
00491     _stack.resize(stacksize);
00492     _alloccallsstacksize = 4;
00493     _callstackdata.resize(_alloccallsstacksize);
00494     _callsstacksize = 0;
00495     _callsstack = &_callstackdata[0];
00496     _stackbase = 0;
00497     _top = 0;
00498     if(!friendvm) {
00499         _roottable = SQTable::Create(_ss(this), 0);
00500         sq_base_register(this);
00501     }
```

```
00502     else {
00503         _roottable = friendvm->_roottable;
00504         _errorhandler = friendvm->_errorhandler;
00505         _debughook = friendvm->_debughook;
00506         _debughook_native = friendvm->_debughook_native;
00507         _debughook_closure = friendvm->_debughook_closure;
00508     }
00509
00510
00511     return true;
00512 }
00513
00526 bool SQVM::StartCall(SQClosure *closure,SQInteger target,SQInteger args,SQInteger stackbase,bool
      tailcall)
00527 {
00528     SQFunctionProto *func = closure->_function;
00529
00530     SQInteger paramssize = func->_nparameters;
00531     const SQInteger newtop = stackbase + func->_stacksize;
00532     SQInteger nargs = args;
00533     if(func->_varparams)
00534     {
00535         paramssize--;
00536         if (nargs < paramssize) {
00537             Raise_Error(_SC("wrong number of parameters (%d passed, at least %d required)"),
00538               (int)nargs, (int)paramssize);
00539             return false;
00540         }
00541
00542         //dumpstack(stackbase);
00543         SQInteger nvargs = nargs - paramssize;
00544         SQArray *arr = SQArray::Create(_ss(this),nvargs);
00545         SQInteger pbase = stackbase+paramssize;
00546         for(SQInteger n = 0; n < nvargs; n++) {
00547             arr->_values[n] = _stack._vals[pbase];
00548             _stack._vals[pbase].Null();
00549             pbase++;
00550
00551         }
00552         _stack._vals[stackbase+paramssize] = arr;
00553         //dumpstack(stackbase);
00554     }
00555     else if (paramssize != nargs) {
00556         SQInteger ndef = func->_ndefaultparams;
00557         SQInteger diff;
00558         if(ndef && nargs < paramssize && (diff = paramssize - nargs) <= ndef) {
00559             for(SQInteger n = ndef - diff; n < ndef; n++) {
00560                 _stack._vals[stackbase + (nargs++)] = closure->_defaultparams[n];
00561             }
00562         }
00563         else {
00564             Raise_Error(_SC("wrong number of parameters (%d passed, %d required)"),
00565               (int)nargs, (int)paramssize);
00566             return false;
00567         }
00568     }
00569
00570     if(closure->_env) {
00571         _stack._vals[stackbase] = closure->_env->_obj;
00572     }
00573
00574     if(!EnterFrame(stackbase, newtop, tailcall)) return false;
00575
00576     ci->_closure  = closure;
00577     ci->_literals = func->_literals;
00578     ci->_ip       = func->_instructions;
00579     ci->_target   = (SQInt32)target;
00580
00581     if (_debughook) {
00582         CallDebugHook(_SC('c'));
00583     }
00584
00585     if (closure->_function->_bgenerator) {
00586         SQFunctionProto *f = closure->_function;
00587         SQGenerator *gen = SQGenerator::Create(_ss(this), closure);
00588         if(!gen->Yield(this,f->_stacksize))
00589             return false;
00590         SQObjectPtr temp;
00591         Return(1, target, temp);
00592         STK(target) = gen;
00593     }
00594
00595
00596     return true;
00597 }
00598
00608 bool SQVM::Return(SQInteger _arg0, SQInteger _arg1, SQObjectPtr &retval)
```

```
00609 {
00610     SQBool    _isroot      = ci->_root;
00611     SQInteger callerbase   = _stackbase - ci->_prevstkbase;
00612
00613     if (_debughook) {
00614         for(SQInteger i=0; i<ci->_ncalls; i++) {
00615             CallDebugHook(_SC('r'));
00616         }
00617     }
00618
00619     SQObjectPtr *dest;
00620     if (_isroot) {
00621         dest = &(retval);
00622     } else if (ci->_target == -1) {
00623         dest = NULL;
00624     } else {
00625         dest = &_stack._vals[callerbase + ci->_target];
00626     }
00627     if (dest) {
00628         if(_arg0 != 0xFF) {
00629             *dest = _stack._vals[_stackbase+_arg1];
00630         }
00631         else {
00632             dest->Null();
00633         }
00634         //*dest = (_arg0 != 0xFF) ? _stack._vals[_stackbase+_arg1] : _null_;
00635     }
00636     LeaveFrame();
00637     return _isroot ? true : false;
00638 }
00639
00644 #define _RET_ON_FAIL(exp) { if(!exp) return false; }
00645
00655 bool SQVM::PLOCAL_INC(SQInteger op,SQObjectPtr &target, SQObjectPtr &a, SQObjectPtr &incr)
00656 {
00657     SQObjectPtr trg;
00658     _RET_ON_FAIL(ARITH_OP( op , trg, a, incr));
00659     target = a;
00660     a = trg;
00661     return true;
00662 }
00663
00677 bool SQVM::DerefInc(SQInteger op,SQObjectPtr &target, SQObjectPtr &self, SQObjectPtr &key, SQObjectPtr
      &incr, bool postfix,SQInteger selfidx)
00678 {
00679     SQObjectPtr tmp, tself = self, tkey = key;
00680     if (!Get(tself, tkey, tmp, 0, selfidx)) { return false; }
00681     _RET_ON_FAIL(ARITH_OP( op , target, tmp, incr))
00682     if (!Set(tself, tkey, target,selfidx)) { return false; }
00683     if (postfix) target = tmp;
00684     return true;
00685 }
00686
00688 #define arg0 (_i_._arg0)
00690 #define sarg0 ((SQInteger)*((const signed char *)&_i_._arg0))
00692 #define arg1 (_i_._arg1)
00694 #define sarg1 (*((const SQInt32 *)&_i_._arg1))
00696 #define arg2 (_i_._arg2)
00698 #define arg3 (_i_._arg3)
00700 #define sarg3 ((SQInteger)*((const signed char *)&_i_._arg3))
00701
00707 SQRESULT SQVM::Suspend()
00708 {
00709     if (_suspended)
00710         return sq_throwerror(this, _SC("cannot suspend an already suspended vm"));
00711     if (_nnativecalls!=2)
00712         return sq_throwerror(this, _SC("cannot suspend through native calls/metamethods"));
00713     return SQ_SUSPEND_FLAG;
00714 }
00715
00716
00721 #define _FINISH(howmuchtojump) {jump = howmuchtojump; return true; }
00735 bool SQVM::FOREACH_OP(SQObjectPtr &o1,SQObjectPtr &o2,SQObjectPtr
00736 &o3,SQObjectPtr &o4,SQInteger SQ_UNUSED_ARG(arg_2),int exitpos,int &jump)
00737 {
00738     SQInteger nrefidx;
00739     switch(sq_type(o1)) {
00740     case OT_TABLE:
00741         if((nrefidx = _table(o1)->Next(false,o4, o2, o3)) == -1) _FINISH(exitpos);
00742         o4 = (SQInteger)nrefidx; _FINISH(1);
00743     case OT_ARRAY:
00744         if((nrefidx = _array(o1)->Next(o4, o2, o3)) == -1) _FINISH(exitpos);
00745         o4 = (SQInteger) nrefidx; _FINISH(1);
00746     case OT_STRING:
00747         if((nrefidx = _string(o1)->Next(o4, o2, o3)) == -1)_FINISH(exitpos);
00748         o4 = (SQInteger)nrefidx; _FINISH(1);
00749     case OT_CLASS:
```

```
00750         if((nrefidx = _class(o1)->Next(o4, o2, o3)) == -1)_FINISH(exitpos);
00751         o4 = (SQInteger)nrefidx; _FINISH(1);
00752     case OT_USERDATA:
00753     case OT_INSTANCE:
00754         if(_delegable(o1)->_delegate) {
00755             SQObjectPtr itr;
00756             SQObjectPtr closure;
00757             if(_delegable(o1)->GetMetaMethod(this, MT_NEXTI, closure)) {
00758                 Push(o1);
00759                 Push(o4);
00760                 if(CallMetaMethod(closure, MT_NEXTI, 2, itr)) {
00761                     o4 = o2 = itr;
00762                     if(sq_type(itr) == OT_NULL) _FINISH(exitpos);
00763                     if(!Get(o1, itr, o3, 0, DONT_FALL_BACK)) {
00764                         Raise_Error(_SC("_nexti returned an invalid idx")); // cloud be changed
00765                         return false;
00766                     }
00767                     _FINISH(1);
00768                 }
00769                 else {
00770                     return false;
00771                 }
00772             }
00773             Raise_Error(_SC("_nexti failed"));
00774             return false;
00775         }
00776         break;
00777     case OT_GENERATOR:
00778         if(_generator(o1)->_state == SQGenerator::eDead) _FINISH(exitpos);
00779         if(_generator(o1)->_state == SQGenerator::eSuspended) {
00780             SQInteger idx = 0;
00781             if(sq_type(o4) == OT_INTEGER) {
00782                 idx = _integer(o4) + 1;
00783             }
00784             o2 = idx;
00785             o4 = idx;
00786             _generator(o1)->Resume(this, o3);
00787             _FINISH(0);
00788         }
00789     default:
00790         Raise_Error(_SC("cannot iterate %s"), GetTypeName(o1));
00791     }
00792     return false; //cannot be hit(just to avoid warnings)
00793 }
00794
00796 #define COND_LITERAL (arg3!=0?ci->_literals[arg1]:STK(arg1))
00797
00799 #define SQ_THROW() { goto exception_trap; }
00800
00802 #define _GUARD(exp) { if(!exp) { SQ_THROW();} }
00803
00813 bool SQVM::CLOSURE_OP(SQObjectPtr &target, SQFunctionProto *func,SQInteger boundtarget)
00814 {
00815     SQInteger nouters;
00816     SQClosure *closure = SQClosure::Create(_ss(this), func,_table(_roottable)->GetWeakRef(OT_TABLE));
00817     if((nouters = func->_noutervalues)) {
00818         for(SQInteger i = 0; i<nouters; i++) {
00819             SQOuterVar &v = func->_outervalues[i];
00820             switch(v._type){
00821             case otLOCAL:
00822                 FindOuter(closure->_outervalues[i], &STK(_integer(v._src)));
00823                 break;
00824             case otOUTER:
00825                 closure->_outervalues[i] = _closure(ci->_closure)->_outervalues[_integer(v._src)];
00826                 break;
00827             }
00828         }
00829     }
00830     SQInteger ndefparams;
00831     if((ndefparams = func->_ndefaultparams)) {
00832         for(SQInteger i = 0; i < ndefparams; i++) {
00833             SQInteger spos = func->_defaultparams[i];
00834             closure->_defaultparams[i] = _stack._vals[_stackbase + spos];
00835         }
00836     }
00837     if (boundtarget != 0xFF) {
00838         SQObjectPtr &val = _stack._vals[_stackbase + boundtarget];
00839         SQObjectType t = sq_type(val);
00840         if (t == OT_TABLE || t == OT_CLASS || t == OT_INSTANCE || t == OT_ARRAY) {
00841             closure->_env = _refcounted(val)->GetWeakRef(t);
00842             __ObjAddRef(closure->_env);
00843         }
00844         else {
00845             Raise_Error(_SC("cannot bind a %s as environment object"), IdType2Name(t));
00846             closure->Release();
00847             return false;
00848         }
```

```
00849     }
00850     target = closure;
00851     return true;
00852
00853 }
00854
00855
00865 bool SQVM::CLASS_OP(SQObjectPtr &target,SQInteger baseclass,SQInteger attributes)
00866 {
00867     SQClass *base = NULL;
00868     SQObjectPtr attrs;
00869     if(baseclass != -1) {
00870         if(sq_type(_stack._vals[_stackbase+baseclass]) != OT_CLASS) { Raise_Error(_SC("trying to
      inherit from a %s"),GetTypeName(_stack._vals[_stackbase+baseclass])); return false; }
00871         base = _class(_stack._vals[_stackbase + baseclass]);
00872     }
00873     if(attributes != MAX_FUNC_STACKSIZE) {
00874         attrs = _stack._vals[_stackbase+attributes];
00875     }
00876     target = SQClass::Create(_ss(this),base);
00877     if(sq_type(_class(target)->_metamethods[MT_INHERITED]) != OT_NULL) {
00878         int nparams = 2;
00879         SQObjectPtr ret;
00880         Push(target); Push(attrs);
00881         if(!Call(_class(target)->_metamethods[MT_INHERITED],nparams,_top - nparams, ret, false)) {
00882             Pop(nparams);
00883             return false;
00884         }
00885         Pop(nparams);
00886     }
00887     _class(target)->_attributes = attrs;
00888     return true;
00889 }
00890
00899 bool SQVM::IsEqual(const SQObjectPtr &o1,const SQObjectPtr &o2,bool &res)
00900 {
00901     SQObjectType t1 = sq_type(o1), t2 = sq_type(o2);
00902     if(t1 == t2) {
00903         if (t1 == OT_FLOAT) {
00904             res = (_float(o1) == _float(o2));
00905         }
00906         else {
00907             res = (_rawval(o1) == _rawval(o2));
00908         }
00909     }
00910     else {
00911         if(sq_isnumeric(o1) && sq_isnumeric(o2)) {
00912             res = (tofloat(o1) == tofloat(o2));
00913         }
00914         else {
00915             res = false;
00916         }
00917     }
00918     return true;
00919 }
00920
00927 bool SQVM::IsFalse(SQObjectPtr &o)
00928 {
00929     if(((sq_type(o) & SQOBJECT_CANBEFALSE)
00930         && ( ((sq_type(o) == OT_FLOAT) && (_float(o) == SQFloat(0.0))) ))
00931 #if !defined(SQUSEDOUBLE) || (defined(SQUSEDOUBLE) && defined(_SQ64))
00932         || (_integer(o) == 0) )  //OT_NULL|OT_INTEGER|OT_BOOL
00933 #else
00934         || (((sq_type(o) != OT_FLOAT) && (_integer(o) == 0))) )  //OT_NULL|OT_INTEGER|OT_BOOL
00935 #endif
00936     {
00937         return true;
00938     }
00939     return false;
00940 }
00944 extern SQInstructionDesc g_InstrDesc[];
00945
00961 bool SQVM::Execute(SQObjectPtr &closure, SQInteger nargs, SQInteger stackbase,SQObjectPtr &outres,
      SQBool raiseerror,ExecutionType et)
00962 {
00963     if ((_nnativecalls + 1) > MAX_NATIVE_CALLS) { Raise_Error(_SC("Native stack overflow")); return
      false; }
00964     _nnativecalls++;
00965     AutoDec ad(&_nnativecalls);
00966     SQInteger traps = 0;
00967     CallInfo *prevci = ci;
00968
00969     switch(et) {
00970         case ET_CALL: {
00971             temp_reg = closure;
00972             if(!StartCall(_closure(temp_reg), _top - nargs, nargs, stackbase, false)) {
00973                 //call the handler if there are no calls in the stack, if not relies on the previous
```

```
    node
00974                 if(ci == NULL) CallErrorHandler(_lasterror);
00975                 return false;
00976             }
00977             if(ci == prevci) {
00978                 outres = STK(_top-nargs);
00979                 return true;
00980             }
00981             ci->_root = SQTrue;
00982                       }
00983             break;
00984         case ET_RESUME_GENERATOR: _generator(closure)->Resume(this, outres); ci->_root = SQTrue; traps
    += ci->_etraps; break;
00985         case ET_RESUME_VM:
00986         case ET_RESUME_THROW_VM:
00987             traps = _suspended_traps;
00988             ci->_root = _suspended_root;
00989             _suspended = SQFalse;
00990             if(et  == ET_RESUME_THROW_VM) { SQ_THROW(); }
00991             break;
00992     }
00993
00994 exception_restore:
00995     //
00996     {
00997         for(;;)
00998         {
00999             const SQInstruction &_i_ = *ci->_ip++;
01000             //dumpstack(_stackbase);
01001             //scprintf("\n[%d] %s %d %d %d
    %d\n",ci->_ip-_closure(ci->_closure)->_function->_instructions,g_InstrDesc[_i_.op].name,arg0,arg1,arg2,arg3);
01002
01003             // 命令のオペコードに基づき、対応する処理を実行するディスパッチループ
01004             switch(_i_.op)
01005             {
01006             case _OP_LINE: if (_debughook) CallDebugHook(_SC('l'),arg1); continue;
01007             case _OP_LOAD: TARGET = ci->_literals[arg1]; continue;
01008             case _OP_LOADINT:
01009 #ifndef _SQ64
01010                 TARGET = (SQInteger)arg1; continue;
01011 #else
01012                 TARGET = (SQInteger)((SQInt32)arg1); continue;
01013 #endif
01014             case _OP_LOADFLOAT: TARGET = *((const SQFloat *)&arg1); continue;
01015             case _OP_DLOAD: TARGET = ci->_literals[arg1]; STK(arg2) = ci->_literals[arg3];continue;
01016             case _OP_TAILCALL:{
01017                 SQObjectPtr &t = STK(arg1);
01018                 if (sq_type(t) == OT_CLOSURE
01019                     && (!_closure(t)->_function->_bgenerator)){
01020                     SQObjectPtr clo = t;
01021                     SQInteger last_top = _top;
01022                     if(_openouters) CloseOuters(&(_stack._vals[_stackbase]));
01023                     for (SQInteger i = 0; i < arg3; i++) STK(i) = STK(arg2 + i);
01024                     _GUARD(StartCall(_closure(clo), ci->_target, arg3, _stackbase, true));
01025                     if (last_top >= _top) {
01026                         _top = last_top;
01027                     }
01028                     continue;
01029                 }
01030                           }
01031             case _OP_CALL: {
01032                     SQObjectPtr clo = STK(arg1);
01033                     switch (sq_type(clo)) {
01034                     case OT_CLOSURE:
01035                         _GUARD(StartCall(_closure(clo), sarg0, arg3, _stackbase+arg2, false));
01036                         continue;
01037                     case OT_NATIVECLOSURE: {
01038                         bool suspend;
01039                         bool tailcall;
01040                         _GUARD(CallNative(_nativeclosure(clo), arg3, _stackbase+arg2, clo,
    (SQInt32)sarg0, suspend, tailcall));
01041                         if(suspend){
01042                             _suspended = SQTrue;
01043                             _suspended_target = sarg0;
01044                             _suspended_root = ci->_root;
01045                             _suspended_traps = traps;
01046                             outres = clo;
01047                             return true;
01048                         }
01049                         if(sarg0 != -1 && !tailcall) {
01050                             STK(arg0) = clo;
01051                         }
01052                                         }
01053                         continue;
01054                     case OT_CLASS:{
01055                         SQObjectPtr inst;
01056                         _GUARD(CreateClassInstance(_class(clo),inst,clo));
```

```
01057                         if(sarg0 != -1) {
01058                             STK(arg0) = inst;
01059                         }
01060                         SQInteger stkbase;
01061                         switch(sq_type(clo)) {
01062                             case OT_CLOSURE:
01063                                 stkbase = _stackbase+arg2;
01064                                 _stack._vals[stkbase] = inst;
01065                                 _GUARD(StartCall(_closure(clo), -1, arg3, stkbase, false));
01066                                 break;
01067                             case OT_NATIVECLOSURE:
01068                                 bool dummy;
01069                                 stkbase = _stackbase+arg2;
01070                                 _stack._vals[stkbase] = inst;
01071                                 _GUARD(CallNative(_nativeclosure(clo), arg3, stkbase, clo, -1, dummy,
      dummy));
01072                                 break;
01073                             default: break; //shutup GCC 4.x
01074                         }
01075                         }
01076                         break;
01077                     case OT_TABLE:
01078                     case OT_USERDATA:
01079                     case OT_INSTANCE:{
01080                         SQObjectPtr closure;
01081                         if(_delegable(clo)->_delegate &&
      _delegable(clo)->GetMetaMethod(this,MT_CALL,closure)) {
01082                             Push(clo);
01083                             for (SQInteger i = 0; i < arg3; i++) Push(STK(arg2 + i));
01084                             if(!CallMetaMethod(closure, MT_CALL, arg3+1, clo)) SQ_THROW();
01085                             if(sarg0 != -1) {
01086                                 STK(arg0) = clo;
01087                             }
01088                             break;
01089                         }
01090
01091                         //Raise_Error(_SC("attempt to call '%s'"), GetTypeName(clo));
01092                         //SQ_THROW();
01093                       }
01094                     default:
01095                         Raise_Error(_SC("attempt to call '%s'"), GetTypeName(clo));
01096                         SQ_THROW();
01097                     }
01098                 }
01099                   continue;
01100             case _OP_PREPCALL:
01101             case _OP_PREPCALLK: {
01102                     SQObjectPtr &key = _i_.op == _OP_PREPCALLK?(ci->_literals)[arg1]:STK(arg1);
01103                     SQObjectPtr &o = STK(arg2);
01104                     if (!Get(o, key, temp_reg,0,arg2)) {
01105                         SQ_THROW();
01106                     }
01107                     STK(arg3) = o;
01108                     _Swap(TARGET,temp_reg);//TARGET = temp_reg;
01109                 }
01110                 continue;
01111             case _OP_GETK:
01112                 if (!Get(STK(arg2), ci->_literals[arg1], temp_reg, 0,arg2)) { SQ_THROW();}
01113                 _Swap(TARGET,temp_reg);//TARGET = temp_reg;
01114                 continue;
01115             case _OP_MOVE: TARGET = STK(arg1); continue;
01116             case _OP_NEWSLOT:
01117                 _GUARD(NewSlot(STK(arg1), STK(arg2), STK(arg3),false));
01118                 if(arg0 != 0xFF) TARGET = STK(arg3);
01119                 continue;
01120             case _OP_DELETE: _GUARD(DeleteSlot(STK(arg1), STK(arg2), TARGET)); continue;
01121             case _OP_SET:
01122                 if (!Set(STK(arg1), STK(arg2), STK(arg3),arg1)) { SQ_THROW(); }
01123                 if (arg0 != 0xFF) TARGET = STK(arg3);
01124                 continue;
01125             case _OP_GET:
01126                 if (!Get(STK(arg1), STK(arg2), temp_reg, 0,arg1)) { SQ_THROW(); }
01127                 _Swap(TARGET,temp_reg);//TARGET = temp_reg;
01128                 continue;
01129             case _OP_EQ:{
01130                 bool res;
01131                 if(!IsEqual(STK(arg2),COND_LITERAL,res)) { SQ_THROW(); }
01132                 TARGET = res?true:false;
01133                 }continue;
01134             case _OP_NE:{
01135                 bool res;
01136                 if(!IsEqual(STK(arg2),COND_LITERAL,res)) { SQ_THROW(); }
01137                 TARGET = (!res)?true:false;
01138                 } continue;
01139             case _OP_ADD: _ARITH_(+,TARGET,STK(arg2),STK(arg1)); continue;
01140             case _OP_SUB: _ARITH_(-,TARGET,STK(arg2),STK(arg1)); continue;
01141             case _OP_MUL: _ARITH_(*,TARGET,STK(arg2),STK(arg1)); continue;
```

```
01142             case _OP_DIV: _ARITH_NOZERO(/,TARGET,STK(arg2),STK(arg1),_SC("division by zero"));
      continue;
01143             case _OP_MOD: ARITH_OP('%',TARGET,STK(arg2),STK(arg1)); continue;
01144             case _OP_BITW:  _GUARD(BW_OP( arg3,TARGET,STK(arg2),STK(arg1))); continue;
01145             case _OP_RETURN:
01146                 if((ci)->_generator) {
01147                     (ci)->_generator->Kill();
01148                 }
01149                 if(Return(arg0, arg1, temp_reg)){
01150                     assert(traps==0);
01151                     //outres = temp_reg;
01152                     _Swap(outres,temp_reg);
01153                     return true;
01154                 }
01155                 continue;
01156             case _OP_LOADNULLS:{ for(SQInt32 n=0; n < arg1; n++) STK(arg0+n).Null(); }continue;
01157             case _OP_LOADROOT:  {
01158                 SQWeakRef *w = _closure(ci->_closure)->_root;
01159                 if(sq_type(w->_obj) != OT_NULL) {
01160                     TARGET = w->_obj;
01161                 } else {
01162                     TARGET = _roottable; //shoud this be like this? or null
01163                 }
01164                                 }
01165                 continue;
01166             case _OP_LOADBOOL: TARGET = arg1?true:false; continue;
01167             case _OP_DMOVE: STK(arg0) = STK(arg1); STK(arg2) = STK(arg3); continue;
01168             case _OP_JMP: ci->_ip += (sarg1); continue;
01169             //case _OP_JNZ: if(!IsFalse(STK(arg0))) ci->_ip+=(sarg1); continue;
01170             case _OP_JCMP:
01171                 _GUARD(CMP_OP((CmpOP)arg3,STK(arg2),STK(arg0),temp_reg));
01172                 if(IsFalse(temp_reg)) ci->_ip+=(sarg1);
01173                 continue;
01174             case _OP_JZ: if(IsFalse(STK(arg0))) ci->_ip+=(sarg1); continue;
01175             case _OP_GETOUTER: {
01176                 SQClosure *cur_cls = _closure(ci->_closure);
01177                 SQOuter *otr = _outer(cur_cls->_outervalues[arg1]);
01178                 TARGET = *(otr->_valptr);
01179                 }
01180             continue;
01181             case _OP_SETOUTER: {
01182                 SQClosure *cur_cls = _closure(ci->_closure);
01183                 SQOuter   *otr = _outer(cur_cls->_outervalues[arg1]);
01184                 *(otr->_valptr) = STK(arg2);
01185                 if(arg0 != 0xFF) {
01186                     TARGET = STK(arg2);
01187                 }
01188                 }
01189             continue;
01190             case _OP_NEWOBJ:
01191                 switch(arg3) {
01192                     case NOT_TABLE: TARGET = SQTable::Create(_ss(this), arg1); continue;
01193                     case NOT_ARRAY: TARGET = SQArray::Create(_ss(this), 0);
      _array(TARGET)->Reserve(arg1); continue;
01194                     case NOT_CLASS: _GUARD(CLASS_OP(TARGET,arg1,arg2)); continue;
01195                     default: assert(0); continue;
01196                 }
01197             case _OP_APPENDARRAY:
01198                 {
01199                     SQObject val;
01200                     val._unVal.raw = 0;
01201                 switch(arg2) {
01202                 case AAT_STACK:
01203                     val = STK(arg1); break;
01204                 case AAT_LITERAL:
01205                     val = ci->_literals[arg1]; break;
01206                 case AAT_INT:
01207                     val._type = OT_INTEGER;
01208 #ifndef _SQ64
01209                     val._unVal.nInteger = (SQInteger)arg1;
01210 #else
01211                     val._unVal.nInteger = (SQInteger)((SQInt32)arg1);
01212 #endif
01213                     break;
01214                 case AAT_FLOAT:
01215                     val._type = OT_FLOAT;
01216                     val._unVal.fFloat = *((const SQFloat *)&arg1);
01217                     break;
01218                 case AAT_BOOL:
01219                     val._type = OT_BOOL;
01220                     val._unVal.nInteger = arg1;
01221                     break;
01222                 default: val._type = OT_INTEGER; assert(0); break;
01223
01224                 }
01225                 _array(STK(arg0))->Append(val); continue;
01226                 }
```

```
01227             case _OP_COMPARITH: {
01228                 SQInteger selfidx = (((SQUnsignedInteger)arg1&0xFFFF0000)»16);
01229                 _GUARD(DerefInc(arg3, TARGET, STK(selfidx), STK(arg2), STK(arg1&0x0000FFFF), false,
      selfidx));
01230                                 }
01231                 continue;
01232             case _OP_INC: {SQObjectPtr o(sarg3); _GUARD(DerefInc('+',TARGET, STK(arg1), STK(arg2), o,
      false, arg1));} continue;
01233             case _OP_INCL: {
01234                 SQObjectPtr &a = STK(arg1);
01235                 if(sq_type(a) == OT_INTEGER) {
01236                     a._unVal.nInteger = _integer(a) + sarg3;
01237                 }
01238                 else {
01239                     SQObjectPtr o(sarg3); //_GUARD(LOCAL_INC('+',TARGET, STK(arg1), o));
01240                     _ARITH_(+,a,a,o);
01241                 }
01242                         } continue;
01243             case _OP_PINC: {SQObjectPtr o(sarg3); _GUARD(DerefInc('+',TARGET, STK(arg1), STK(arg2), o,
      true, arg1));} continue;
01244             case _OP_PINCL: {
01245                 SQObjectPtr &a = STK(arg1);
01246                 if(sq_type(a) == OT_INTEGER) {
01247                     TARGET = a;
01248                     a._unVal.nInteger = _integer(a) + sarg3;
01249                 }
01250                 else {
01251                     SQObjectPtr o(sarg3); _GUARD(PLOCAL_INC('+',TARGET, STK(arg1), o));
01252                 }
01253
01254                         } continue;
01255             case _OP_CMP:   _GUARD(CMP_OP((CmpOP)arg3,STK(arg2),STK(arg1),TARGET))  continue;
01256             case _OP_EXISTS: TARGET = Get(STK(arg1), STK(arg2), temp_reg, GET_FLAG_DO_NOT_RAISE_ERROR
      | GET_FLAG_RAW, DONT_FALL_BACK) ? true : false; continue;
01257             case _OP_INSTANCEOF:
01258                 if(sq_type(STK(arg1)) != OT_CLASS)
01259                 {Raise_Error(_SC("cannot apply instanceof between a %s and a
      %s"),GetTypeName(STK(arg1)),GetTypeName(STK(arg2))); SQ_THROW();}
01260                 TARGET = (sq_type(STK(arg2)) == OT_INSTANCE) ?
      (_instance(STK(arg2))->InstanceOf(_class(STK(arg1)))?true:false) : false;
01261                 continue;
01262             case _OP_AND:
01263                 if(IsFalse(STK(arg2))) {
01264                     TARGET = STK(arg2);
01265                     ci->_ip += (sarg1);
01266                 }
01267                 continue;
01268             case _OP_OR:
01269                 if(!IsFalse(STK(arg2))) {
01270                     TARGET = STK(arg2);
01271                     ci->_ip += (sarg1);
01272                 }
01273                 continue;
01274             case _OP_NEG: _GUARD(NEG_OP(TARGET,STK(arg1))); continue;
01275             case _OP_NOT: TARGET = IsFalse(STK(arg1)); continue;
01276             case _OP_BWNOT:
01277                 if(sq_type(STK(arg1)) == OT_INTEGER) {
01278                     SQInteger t = _integer(STK(arg1));
01279                     TARGET = SQInteger(~t);
01280                     continue;
01281                 }
01282                 Raise_Error(_SC("attempt to perform a bitwise op on a %s"), GetTypeName(STK(arg1)));
01283                 SQ_THROW();
01284             case _OP_CLOSURE: {
01285                 SQClosure *c = ci->_closure._unVal.pClosure;
01286                 SQFunctionProto *fp = c->_function;
01287                 if(!CLOSURE_OP(TARGET,fp->_functions[arg1]._unVal.pFunctionProto,arg2)) { SQ_THROW();
      }
01288                 continue;
01289             }
01290             case _OP_YIELD:{
01291                 if(ci->_generator) {
01292                     if(sarg1 != MAX_FUNC_STACKSIZE) temp_reg = STK(arg1);
01293                     if (_openouters) CloseOuters(&_stack._vals[_stackbase]);
01294                     _GUARD(ci->_generator->Yield(this,arg2));
01295                     traps -= ci->_etraps;
01296                     if(sarg1 != MAX_FUNC_STACKSIZE) _Swap(STK(arg1),temp_reg);//STK(arg1) = temp_reg;
01297                 }
01298                 else { Raise_Error(_SC("trying to yield a '%s',only genenerator can be yielded"),
      GetTypeName(ci->_generator)); SQ_THROW();}
01299                 if(Return(arg0, arg1, temp_reg)){
01300                     assert(traps == 0);
01301                     outres = temp_reg;
01302                     return true;
01303                 }
01304
01305                 }
```

```
01306                 continue;
01307             case _OP_RESUME:
01308                 if(sq_type(STK(arg1)) != OT_GENERATOR){ Raise_Error(_SC("trying to resume a '%s',only
      genenerator can be resumed"), GetTypeName(STK(arg1))); SQ_THROW();}
01309                 _GUARD(_generator(STK(arg1))->Resume(this, TARGET));
01310                 traps += ci->_etraps;
01311                 continue;
01312             case _OP_FOREACH:{ int tojump;
01313                 _GUARD(FOREACH_OP(STK(arg0),STK(arg2),STK(arg2+1),STK(arg2+2),arg2,sarg1,tojump));
01314                 ci->_ip += tojump; }
01315                 continue;
01316             case _OP_POSTFOREACH:
01317                 assert(sq_type(STK(arg0)) == OT_GENERATOR);
01318                 if(_generator(STK(arg0))->_state == SQGenerator::eDead)
01319                     ci->_ip += (sarg1 - 1);
01320                 continue;
01321             case _OP_CLONE: _GUARD(Clone(STK(arg1), TARGET)); continue;
01322             case _OP_TYPEOF: _GUARD(TypeOf(STK(arg1), TARGET)) continue;
01323             case _OP_PUSHTRAP:{
01324                 SQInstruction *_iv = _closure(ci->_closure)->_function->_instructions;
01325                 _etraps.push_back(SQExceptionTrap(_top,_stackbase, &_iv[(ci->_ip-_iv)+arg1], arg0));
      traps++;
01326                 ci->_etraps++;
01327                                 }
01328                 continue;
01329             case _OP_POPTRAP: {
01330                 for(SQInteger i = 0; i < arg0; i++) {
01331                     _etraps.pop_back(); traps--;
01332                     ci->_etraps--;
01333                 }
01334                                 }
01335                 continue;
01336             case _OP_THROW: Raise_Error(TARGET); SQ_THROW(); continue;
01337             case _OP_NEWSLOTA:
01338                 _GUARD(NewSlotA(STK(arg1),STK(arg2),STK(arg3),(arg0&NEW_SLOT_ATTRIBUTES_FLAG) ?
      STK(arg2-1) : SQObjectPtr(),(arg0&NEW_SLOT_STATIC_FLAG)?true:false,false));
01339                 continue;
01340             case _OP_GETBASE:{
01341                 SQClosure *clo = _closure(ci->_closure);
01342                 if(clo->_base) {
01343                     TARGET = clo->_base;
01344                 }
01345                 else {
01346                     TARGET.Null();
01347                 }
01348                 continue;
01349             }
01350             case _OP_CLOSE:
01351                 if(_openouters) CloseOuters(&(STK(arg1)));
01352                 continue;
01353             }
01354
01355         }
01356     }
01357 exception_trap:
01358     {
01359         // 例外処理ブロック: エラー発生時にここにジャンプする
01360         SQObjectPtr currerror = _lasterror;
01361 //      dumpstack(_stackbase);
01362 //      SQInteger n = 0;
01363         SQInteger last_top = _top;
01364
01365         if(_ss(this)->_notifyallexceptions || (!traps && raiseerror)) CallErrorHandler(currerror);
01366
01367         while( ci ) {
01368             if(ci->_etraps > 0) {
01369                 // 例外トラップ（try-catch）が見つかった場合
01370                 SQExceptionTrap &et = _etraps.top();
01371                 ci->_ip = et._ip; // IP をハンドラのアドレスに設定
01372                 _top = et._stacksize;
01373                 _stackbase = et._stackbase;
01374                 _stack._vals[_stackbase + et._extarget] = currerror; // エラーオブジェクトをスタックに積む
01375                 _etraps.pop_back(); traps--; ci->_etraps--;
01376                 while(last_top >= _top) _stack._vals[last_top--].Null();
01377                 goto exception_restore; // 実行ループに復帰
01378             }
01379             else if (_debughook) {
01380                     //notify debugger of a "return"
01381                     //even if it really an exception unwinding the stack
01382                     for(SQInteger i = 0; i < ci->_ncalls; i++) {
01383                         CallDebugHook(_SC('r'));
01384                     }
01385             }
01386             if(ci->_generator) ci->_generator->Kill();
01387             bool mustbreak = ci && ci->_root;
01388             LeaveFrame(); // 現在のフレームを破棄してスタックを巻き戻す
01389             if(mustbreak) break;
```

```
01390         }
01391
01392         _lasterror = currerror;
01393         return false;
01394     }
01395     assert(0);
01396 }
01397
01406 bool SQVM::CreateClassInstance(SQClass *theclass, SQObjectPtr &inst, SQObjectPtr &constructor)
01407 {
01408     inst = theclass->CreateInstance();
01409     if(!theclass->GetConstructor(constructor)) {
01410         constructor.Null();
01411     }
01412     return true;
01413 }
01414
01421 void SQVM::CallErrorHandler(SQObjectPtr &error)
01422 {
01423     if(sq_type(_errorhandler) != OT_NULL) {
01424         SQObjectPtr out;
01425         Push(_roottable); Push(error);
01426         Call(_errorhandler, 2, _top-2, out,SQFalse);
01427         Pop(2);
01428     }
01429 }
01430
01431
01439 void SQVM::CallDebugHook(SQInteger type,SQInteger forcedline)
01440 {
01441     _debughook = false;
01442     SQFunctionProto *func=_closure(ci->_closure)->_function;
01443     if(_debughook_native) {
01444         const SQChar *src = sq_type(func->_sourcename) ==
      OT_STRING?_stringval(func->_sourcename):NULL;
01445         const SQChar *fname = sq_type(func->_name) == OT_STRING?_stringval(func->_name):NULL;
01446         SQInteger line = forcedline?forcedline:func->GetLine(ci->_ip);
01447         _debughook_native(this,type,src,line,fname);
01448     }
01449     else {
01450         SQObjectPtr temp_reg;
01451         SQInteger nparams=5;
01452         Push(_roottable); Push(type); Push(func->_sourcename);
      Push(forcedline?forcedline:func->GetLine(ci->_ip)); Push(func->_name);
01453         Call(_debughook_closure,nparams,_top-nparams,temp_reg,SQFalse);
01454         Pop(nparams);
01455     }
01456     _debughook = true;
01457 }
01458
01473 bool SQVM::CallNative(SQNativeClosure *nclosure, SQInteger nargs, SQInteger newbase, SQObjectPtr
      &retval, SQInt32 target,bool &suspend, bool &tailcall)
01474 {
01475     SQInteger nparamscheck = nclosure->_nparamscheck;
01476     SQInteger newtop = newbase + nargs + nclosure->_noutervalues;
01477
01478     if (_nnativecalls + 1 > MAX_NATIVE_CALLS) {
01479         Raise_Error(_SC("Native stack overflow"));
01480         return false;
01481     }
01482
01483     if(nparamscheck && (((nparamscheck > 0) && (nparamscheck != nargs)) ||
01484         ((nparamscheck < 0) && (nargs < (-nparamscheck))))
01485     {
01486         Raise_Error(_SC("wrong number of parameters"));
01487         return false;
01488     }
01489
01490     SQInteger tcs;
01491     SQIntVec &tc = nclosure->_typecheck;
01492     if((tcs = tc.size())) {
01493         for(SQInteger i = 0; i < nargs && i < tcs; i++) {
01494             if((tc._vals[i] != -1) && !(sq_type(_stack._vals[newbase+i]) & tc._vals[i])) {
01495                 Raise_ParamTypeError(i,tc._vals[i], sq_type(_stack._vals[newbase+i]));
01496                 return false;
01497             }
01498         }
01499     }
01500
01501     if(!EnterFrame(newbase, newtop, false)) return false;
01502     ci->_closure  = nclosure;
01503     ci->_target = target;
01504
01505     SQInteger outers = nclosure->_noutervalues;
01506     for (SQInteger i = 0; i < outers; i++) {
01507         _stack._vals[newbase+nargs+i] = nclosure->_outervalues[i];
01508     }
```

```
01509     if(nclosure->_env) {
01510         _stack._vals[newbase] = nclosure->_env->_obj;
01511     }
01512
01513     _nnativecalls++;
01514     SQInteger ret = (nclosure->_function)(this);
01515     _nnativecalls--;
01516
01517     suspend = false;
01518     tailcall = false;
01519     if (ret == SQ_TAILCALL_FLAG) {
01520         tailcall = true;
01521         return true;
01522     }
01523     else if (ret == SQ_SUSPEND_FLAG) {
01524         suspend = true;
01525     }
01526     else if (ret < 0) {
01527         LeaveFrame();
01528         Raise_Error(_lasterror);
01529         return false;
01530     }
01531     if(ret) {
01532         retval = _stack._vals[_top-1];
01533     }
01534     else {
01535         retval.Null();
01536     }
01537     //retval = ret ? _stack._vals[_top-1] : _null_;
01538     LeaveFrame();
01539     return true;
01540 }
01541
01551 bool SQVM::TailCall(SQClosure *closure, SQInteger parambase,SQInteger nparams)
01552 {
01553     SQInteger last_top = _top;
01554     SQObjectPtr clo = closure;
01555     if (ci->_root)
01556     {
01557         Raise_Error("root calls cannot invoke tailcalls");
01558         return false;
01559     }
01560     for (SQInteger i = 0; i < nparams; i++) STK(i) = STK(parambase + i);
01561     bool ret = StartCall(closure, ci->_target, nparams, _stackbase, true);
01562     if (last_top >= _top) {
01563         _top = last_top;
01564     }
01565     return ret;
01566 }
01567
01569 #define FALLBACK_OK         0
01571 #define FALLBACK_NO_MATCH   1
01573 #define FALLBACK_ERROR      2
01574
01586 bool SQVM::Get(const SQObjectPtr &self, const SQObjectPtr &key, SQObjectPtr &dest, SQUnsignedInteger
      getflags, SQInteger selfidx)
01587 {
01588     switch(sq_type(self)){
01589     case OT_TABLE:
01590         if(_table(self)->Get(key,dest))return true;
01591         break;
01592     case OT_ARRAY:
01593         if (sq_isnumeric(key)) { if (_array(self)->Get(tointeger(key), dest)) { return true; } if
      ((getflags & GET_FLAG_DO_NOT_RAISE_ERROR) == 0) Raise_IdxError(key); return false; }
01594         break;
01595     case OT_INSTANCE:
01596         if(_instance(self)->Get(key,dest)) return true;
01597         break;
01598     case OT_CLASS:
01599         if(_class(self)->Get(key,dest)) return true;
01600         break;
01601     case OT_STRING:
01602         if(sq_isnumeric(key)){
01603             SQInteger n = tointeger(key);
01604             SQInteger len = _string(self)->_len;
01605             if (n < 0) { n += len; }
01606             if (n >= 0 && n < len) {
01607                 dest = SQInteger(_stringval(self)[n]);
01608                 return true;
01609             }
01610             if ((getflags & GET_FLAG_DO_NOT_RAISE_ERROR) == 0) Raise_IdxError(key);
01611             return false;
01612         }
01613         break;
01614     default:break; //shut up compiler
01615     }
01616     if ((getflags & GET_FLAG_RAW) == 0) {
```

```
01617         switch(FallBackGet(self,key,dest)) {
01618             case FALLBACK_OK: return true; //okie
01619             case FALLBACK_NO_MATCH: break; //keep falling back
01620             case FALLBACK_ERROR: return false; // the metamethod failed
01621         }
01622         if(InvokeDefaultDelegate(self,key,dest)) {
01623             return true;
01624         }
01625     }
01626 //#ifdef ROOT_FALLBACK
01627     if(selfidx == 0) {
01628         SQWeakRef *w = _closure(ci->_closure)->_root;
01629         if(sq_type(w->_obj) != OT_NULL)
01630         {
01631             if(Get(*((const SQObjectPtr *)&w->_obj),key,dest,0,DONT_FALL_BACK)) return true;
01632         }
01633 
01634     }
01635 //#endif
01636     if ((getflags & GET_FLAG_DO_NOT_RAISE_ERROR) == 0) Raise_IdxError(key);
01637     return false;
01638 }
01639 
01649 bool SQVM::InvokeDefaultDelegate(const SQObjectPtr &self,const SQObjectPtr &key,SQObjectPtr &dest)
01650 {
01651     SQTable *ddel = NULL;
01652     switch(sq_type(self)) {
01653         case OT_CLASS: ddel = _class_ddel; break;
01654         case OT_TABLE: ddel = _table_ddel; break;
01655         case OT_ARRAY: ddel = _array_ddel; break;
01656         case OT_STRING: ddel = _string_ddel; break;
01657         case OT_INSTANCE: ddel = _instance_ddel; break;
01658         case OT_INTEGER:case OT_FLOAT:case OT_BOOL: ddel = _number_ddel; break;
01659         case OT_GENERATOR: ddel = _generator_ddel; break;
01660         case OT_CLOSURE: case OT_NATIVECLOSURE: ddel = _closure_ddel; break;
01661         case OT_THREAD: ddel = _thread_ddel; break;
01662         case OT_WEAKREF: ddel = _weakref_ddel; break;
01663         default: return false;
01664     }
01665     return  ddel->Get(key,dest);
01666 }
01667 
01668 
01678 SQInteger SQVM::FallBackGet(const SQObjectPtr &self,const SQObjectPtr &key,SQObjectPtr &dest)
01679 {
01680     switch(sq_type(self)){
01681     case OT_TABLE:
01682     case OT_USERDATA:
01683         //delegation
01684         if(_delegable(self)->_delegate) {
01685             if(Get(SQObjectPtr(_delegable(self)->_delegate),key,dest,
      GET_FLAG_DO_NOT_RAISE_ERROR,DONT_FALL_BACK)) return FALLBACK_OK;
01686         }
01687         else {
01688             return FALLBACK_NO_MATCH;
01689         }
01690         //go through
01691     case OT_INSTANCE: {
01692         SQObjectPtr closure;
01693         if(_delegable(self)->GetMetaMethod(this, MT_GET, closure)) {
01694             Push(self);Push(key);
01695             _nmetamethodscall++;
01696             AutoDec ad(&_nmetamethodscall);
01697             if(Call(closure, 2, _top - 2, dest, SQFalse)) {
01698                 Pop(2);
01699                 return FALLBACK_OK;
01700             }
01701             else {
01702                 Pop(2);
01703                 if(sq_type(_lasterror) != OT_NULL) { //NULL means "clean failure" (not found)
01704                     return FALLBACK_ERROR;
01705                 }
01706             }
01707         }
01708                     }
01709         break;
01710     default: break;//shutup GCC 4.x
01711     }
01712     // no metamethod or no fallback type
01713     return FALLBACK_NO_MATCH;
01714 }
01715 
01726 bool SQVM::Set(const SQObjectPtr &self,const SQObjectPtr &key,const SQObjectPtr &val,SQInteger
      selfidx)
01727 {
01728     switch(sq_type(self)){
01729     case OT_TABLE:
```

```
01730         if(_table(self)->Set(key,val)) return true;
01731         break;
01732     case OT_INSTANCE:
01733         if(_instance(self)->Set(key,val)) return true;
01734         break;
01735     case OT_ARRAY:
01736         if(!sq_isnumeric(key)) { Raise_Error(_SC("indexing %s with
      %s"),GetTypeName(self),GetTypeName(key)); return false; }
01737         if(!_array(self)->Set(tointeger(key),val)) {
01738             Raise_IdxError(key);
01739             return false;
01740         }
01741         return true;
01742     case OT_USERDATA: break; // must fall back
01743     default:
01744         Raise_Error(_SC("trying to set '%s'"),GetTypeName(self));
01745         return false;
01746     }
01747
01748     switch(FallBackSet(self,key,val)) {
01749         case FALLBACK_OK: return true; //okie
01750         case FALLBACK_NO_MATCH: break; //keep falling back
01751         case FALLBACK_ERROR: return false; // the metamethod failed
01752     }
01753     if(selfidx == 0) {
01754         if(_table(_roottable)->Set(key,val))
01755             return true;
01756     }
01757     Raise_IdxError(key);
01758     return false;
01759 }
01760
01770 SQInteger SQVM::FallBackSet(const SQObjectPtr &self,const SQObjectPtr &key,const SQObjectPtr &val)
01771 {
01772     switch(sq_type(self)) {
01773     case OT_TABLE:
01774         if(_table(self)->_delegate) {
01775             if(Set(_table(self)->_delegate,key,val,DONT_FALL_BACK)) return FALLBACK_OK;
01776         }
01777         //keps on going
01778     case OT_INSTANCE:
01779     case OT_USERDATA:{
01780         SQObjectPtr closure;
01781         SQObjectPtr t;
01782         if(_delegable(self)->GetMetaMethod(this, MT_SET, closure)) {
01783             Push(self);Push(key);Push(val);
01784             _nmetamethodscall++;
01785             AutoDec ad(&_nmetamethodscall);
01786             if(Call(closure, 3, _top - 3, t, SQFalse)) {
01787                 Pop(3);
01788                 return FALLBACK_OK;
01789             }
01790             else {
01791                 Pop(3);
01792                 if(sq_type(_lasterror) != OT_NULL) { //NULL means "clean failure" (not found)
01793                     return FALLBACK_ERROR;
01794                 }
01795             }
01796         }
01797                     }
01798         break;
01799         default: break;//shutup GCC 4.x
01800     }
01801     // no metamethod or no fallback type
01802     return FALLBACK_NO_MATCH;
01803 }
01804
01813 bool SQVM::Clone(const SQObjectPtr &self,SQObjectPtr &target)
01814 {
01815     SQObjectPtr temp_reg;
01816     SQObjectPtr newobj;
01817     switch(sq_type(self)){
01818     case OT_TABLE:
01819         newobj = _table(self)->Clone();
01820         goto cloned_mt;
01821     case OT_INSTANCE: {
01822         newobj = _instance(self)->Clone(_ss(this));
01823 cloned_mt:
01824         SQObjectPtr closure;
01825         if(_delegable(newobj)->_delegate && _delegable(newobj)->GetMetaMethod(this,MT_CLONED,closure))
      {
01826             Push(newobj);
01827             Push(self);
01828             if(!CallMetaMethod(closure,MT_CLONED,2,temp_reg))
01829                 return false;
01830         }
01831         }
```

```
01832         target = newobj;
01833         return true;
01834     case OT_ARRAY:
01835         target = _array(self)->Clone();
01836         return true;
01837     default:
01838         Raise_Error(_SC("cloning a %s"), GetTypeName(self));
01839         return false;
01840     }
01841 }
01842
01855 bool SQVM::NewSlotA(const SQObjectPtr &self,const SQObjectPtr &key,const SQObjectPtr &val,const
      SQObjectPtr &attrs,bool bstatic,bool raw)
01856 {
01857     if(sq_type(self) != OT_CLASS) {
01858         Raise_Error(_SC("object must be a class"));
01859         return false;
01860     }
01861     SQClass *c = _class(self);
01862     if(!raw) {
01863         SQObjectPtr &mm = c->_metamethods[MT_NEWMEMBER];
01864         if(sq_type(mm) != OT_NULL ) {
01865             Push(self); Push(key); Push(val);
01866             Push(attrs);
01867             Push(bstatic);
01868             return CallMetaMethod(mm,MT_NEWMEMBER,5,temp_reg);
01869         }
01870     }
01871     if(!NewSlot(self, key, val,bstatic))
01872         return false;
01873     if(sq_type(attrs) != OT_NULL) {
01874         c->SetAttributes(key,attrs);
01875     }
01876     return true;
01877 }
01878
01891 bool SQVM::NewSlot(const SQObjectPtr &self,const SQObjectPtr &key,const SQObjectPtr &val,bool bstatic)
01892 {
01893     if(sq_type(key) == OT_NULL) { Raise_Error(_SC("null cannot be used as index")); return false; }
01894     switch(sq_type(self)) {
01895     case OT_TABLE: {
01896         bool rawcall = true;
01897         if(_table(self)->_delegate) {
01898             SQObjectPtr res;
01899             if(!_table(self)->Get(key,res)) {
01900                 SQObjectPtr closure;
01901                 if(_delegable(self)->_delegate &&
      _delegable(self)->GetMetaMethod(this,MT_NEWSLOT,closure)) {
01902                     Push(self);Push(key);Push(val);
01903                     if(!CallMetaMethod(closure,MT_NEWSLOT,3,res)) {
01904                         return false;
01905                     }
01906                     rawcall = false;
01907                 }
01908                 else {
01909                     rawcall = true;
01910                 }
01911             }
01912         }
01913         if(rawcall) _table(self)->NewSlot(key,val); //cannot fail
01914
01915         break;}
01916     case OT_INSTANCE: {
01917         SQObjectPtr res;
01918         SQObjectPtr closure;
01919         if(_delegable(self)->_delegate && _delegable(self)->GetMetaMethod(this,MT_NEWSLOT,closure)) {
01920             Push(self);Push(key);Push(val);
01921             if(!CallMetaMethod(closure,MT_NEWSLOT,3,res)) {
01922                 return false;
01923             }
01924             break;
01925         }
01926         Raise_Error(_SC("class instances do not support the new slot operator"));
01927         return false;
01928         break;}
01929     case OT_CLASS:
01930         if(!_class(self)->NewSlot(_ss(this),key,val,bstatic)) {
01931             if(_class(self)->_locked) {
01932                 Raise_Error(_SC("trying to modify a class that has already been instantiated"));
01933                 return false;
01934             }
01935             else {
01936                 SQObjectPtr oval = PrintObjVal(key);
01937                 Raise_Error(_SC("the property '%s' already exists"),_stringval(oval));
01938                 return false;
01939             }
01940         }
```

```
01941         break;
01942     default:
01943         Raise_Error(_SC("indexing %s with %s"),GetTypeName(self),GetTypeName(key));
01944         return false;
01945         break;
01946     }
01947     return true;
01948 }
01949
01950
01951
01961 bool SQVM::DeleteSlot(const SQObjectPtr &self,const SQObjectPtr &key,SQObjectPtr &res)
01962 {
01963     switch(sq_type(self)) {
01964     case OT_TABLE:
01965     case OT_INSTANCE:
01966     case OT_USERDATA: {
01967         SQObjectPtr t;
01968         //bool handled = false;
01969         SQObjectPtr closure;
01970         if(_delegable(self)->_delegate && _delegable(self)->GetMetaMethod(this,MT_DELSLOT,closure)) {
01971             Push(self);Push(key);
01972             return CallMetaMethod(closure,MT_DELSLOT,2,res);
01973         }
01974         else {
01975             if(sq_type(self) == OT_TABLE) {
01976                 if(_table(self)->Get(key,t)) {
01977                     _table(self)->Remove(key);
01978                 }
01979                 else {
01980                     Raise_IdxError((const SQObject &)key);
01981                     return false;
01982                 }
01983             }
01984             else {
01985                 Raise_Error(_SC("cannot delete a slot from %s"),GetTypeName(self));
01986                 return false;
01987             }
01988         }
01989         res = t;
01990                 }
01991         break;
01992     default:
01993         Raise_Error(_SC("attempt to delete a slot from a %s"),GetTypeName(self));
01994         return false;
01995     }
01996     return true;
01997 }
01998
02010 bool SQVM::Call(SQObjectPtr &closure,SQInteger nparams,SQInteger stackbase,SQObjectPtr &outres,SQBool
      raiseerror)
02011 {
02012 #ifdef _DEBUG
02013 SQInteger prevstackbase = _stackbase;
02014 #endif
02015     switch(sq_type(closure)) {
02016     case OT_CLOSURE:
02017         return Execute(closure, nparams, stackbase, outres, raiseerror);
02018         break;
02019     case OT_NATIVECLOSURE:{
02020         bool dummy;
02021         return CallNative(_nativeclosure(closure), nparams, stackbase, outres, -1, dummy, dummy);
02022
02023                         }
02024         break;
02025     case OT_CLASS: {
02026         SQObjectPtr constr;
02027         SQObjectPtr temp;
02028         CreateClassInstance(_class(closure),outres,constr);
02029         SQObjectType ctype = sq_type(constr);
02030         if (ctype == OT_NATIVECLOSURE || ctype == OT_CLOSURE) {
02031             _stack[stackbase] = outres;
02032             return Call(constr,nparams,stackbase,temp,raiseerror);
02033         }
02034         return true;
02035                 }
02036         break;
02037     default:
02038         Raise_Error(_SC("attempt to call '%s'"), GetTypeName(closure));
02039         return false;
02040     }
02041 #ifdef _DEBUG
02042     if(!_suspended) {
02043         assert(_stackbase == prevstackbase);
02044     }
02045 #endif
02046     return true;
```

```
02047 }
02048
02059 bool SQVM::CallMetaMethod(SQObjectPtr &closure,SQMetaMethod SQ_UNUSED_ARG(mm),SQInteger
      nparams,SQObjectPtr &outres)
02060 {
02061     //SQObjectPtr closure;
02062
02063     _nmetamethodscall++;
02064     if(Call(closure, nparams, _top - nparams, outres, SQFalse)) {
02065         _nmetamethodscall--;
02066         Pop(nparams);
02067         return true;
02068     }
02069     _nmetamethodscall--;
02070     //}
02071     Pop(nparams);
02072     return false;
02073 }
02074
02083 void SQVM::FindOuter(SQObjectPtr &target, SQObjectPtr *stackindex)
02084 {
02085     SQOuter **pp = &_openouters;
02086     SQOuter *p;
02087     SQOuter *otr;
02088
02089     while ((p = *pp) != NULL && p->_valptr >= stackindex) {
02090         if (p->_valptr == stackindex) {
02091             target = SQObjectPtr(p);
02092             return;
02093         }
02094         pp = &p->_next;
02095     }
02096     otr = SQOuter::Create(_ss(this), stackindex);
02097     otr->_next = *pp;
02098     otr->_idx  = (stackindex - _stack._vals);
02099     __ObjAddRef(otr);
02100     *pp = otr;
02101     target = SQObjectPtr(otr);
02102 }
02103
02114 bool SQVM::EnterFrame(SQInteger newbase, SQInteger newtop, bool tailcall)
02115 {
02116     if( !tailcall ) {
02117         if( _callsstacksize == _alloccallsstacksize ) {
02118             GrowCallStack();
02119         }
02120         ci = &_callsstack[_callsstacksize++];
02121         ci->_prevstkbase = (SQInt32)(newbase - _stackbase);
02122         ci->_prevtop = (SQInt32)(_top - _stackbase);
02123         ci->_etraps = 0;
02124         ci->_ncalls = 1;
02125         ci->_generator = NULL;
02126         ci->_root = SQFalse;
02127     }
02128     else {
02129         ci->_ncalls++;
02130     }
02131
02132     _stackbase = newbase;
02133     _top = newtop;
02134     if(newtop + MIN_STACK_OVERHEAD > (SQInteger)_stack.size()) {
02135         if(_nmetamethodscall) {
02136             Raise_Error(_SC("stack overflow, cannot resize stack while in a metamethod"));
02137             return false;
02138         }
02139         _stack.resize(newtop + (MIN_STACK_OVERHEAD « 2));
02140         RelocateOuters();
02141     }
02142     return true;
02143 }
02144
02150 void SQVM::LeaveFrame() {
02151     SQInteger last_top = _top;
02152     SQInteger last_stackbase = _stackbase;
02153     SQInteger css = --_callsstacksize;
02154
02155     /* First clean out the call stack frame */
02156     ci->_closure.Null();
02157     _stackbase -= ci->_prevstkbase;
02158     _top = _stackbase + ci->_prevtop;
02159     ci = (css) ? &_callsstack[css-1] : NULL;
02160
02161     if(_openouters) CloseOuters(&(_stack._vals[last_stackbase]));
02162     while (last_top >= _top) {
02163         _stack._vals[last_top--].Null();
02164     }
02165 }
```

```
02166
02172 void SQVM::RelocateOuters()
02173 {
02174     SQOuter *p = _openouters;
02175     while (p) {
02176         p->_valptr = _stack._vals + p->_idx;
02177         p = p->_next;
02178     }
02179 }
02180
02188 void SQVM::CloseOuters(SQObjectPtr *stackindex) {
02189   SQOuter *p;
02190   while ((p = _openouters) != NULL && p->_valptr >= stackindex) {
02191     p->_value = *(p->_valptr);
02192     p->_valptr = &p->_value;
02193     _openouters = p->_next;
02194     __ObjRelease(p);
02195   }
02196 }
02197
02202 void SQVM::Remove(SQInteger n) {
02203     n = (n >= 0)?n + _stackbase - 1:_top + n;
02204     for(SQInteger i = n; i < _top; i++){
02205         _stack[i] = _stack[i+1];
02206     }
02207     _stack[_top].Null();
02208     _top--;
02209 }
02210
02214 void SQVM::Pop() {
02215     _stack[--_top].Null();
02216 }
02217
02222 void SQVM::Pop(SQInteger n) {
02223     for(SQInteger i = 0; i < n; i++){
02224         _stack[--_top].Null();
02225     }
02226 }
02227
02229 void SQVM::PushNull() { _stack[_top++].Null(); }
02231 void SQVM::Push(const SQObjectPtr &o) { _stack[_top++] = o; }
02233 SQObjectPtr &SQVM::Top() { return _stack[_top-1]; }
02235 SQObjectPtr &SQVM::PopGet() { return _stack[--_top]; }
02237 SQObjectPtr &SQVM::GetUp(SQInteger n) { return _stack[_top+n]; }
02239 SQObjectPtr &SQVM::GetAt(SQInteger n) { return _stack[n]; }
02240
02241 #ifdef _DEBUG_DUMP
02247 void SQVM::dumpstack(SQInteger stackbase,bool dumpall)
02248 {
02249     SQInteger size=dumpall?_stack.size():_top;
02250     SQInteger n=0;
02251     scprintf(_SC("\n»»stack dump««\n"));
02252     CallInfo &ci=_callsstack[_callsstacksize-1];
02253     scprintf(_SC("IP: %p\n"),ci._ip);
02254     scprintf(_SC("prev stack base: %d\n"),ci._prevstkbase);
02255     scprintf(_SC("prev top: %d\n"),ci._prevtop);
02256     for(SQInteger i=0;i<size;i++){
02257         SQObjectPtr &obj=_stack[i];
02258         if(stackbase==i)scprintf(_SC(">"));else scprintf(_SC(" "));
02259         scprintf(_SC("[" _PRINT_INT_FMT "]:"),n);
02260         switch(sq_type(obj)){
02261         case OT_FLOAT:          scprintf(_SC("FLOAT %.3f"),_float(obj));break;
02262         case OT_INTEGER:        scprintf(_SC("INTEGER " _PRINT_INT_FMT),_integer(obj));break;
02263         case OT_BOOL:           scprintf(_SC("BOOL %s"),_integer(obj)?"true":"false");break;
02264         case OT_STRING:         scprintf(_SC("STRING %s"),_stringval(obj));break;
02265         case OT_NULL:           scprintf(_SC("NULL"));  break;
02266         case OT_TABLE:          scprintf(_SC("TABLE
      %p[%p]"),_table(obj),_table(obj)->_delegate);break;
02267         case OT_ARRAY:          scprintf(_SC("ARRAY %p"),_array(obj));break;
02268         case OT_CLOSURE:        scprintf(_SC("CLOSURE [%p]"),_closure(obj));break;
02269         case OT_NATIVECLOSURE:  scprintf(_SC("NATIVECLOSURE"));break;
02270         case OT_USERDATA:       scprintf(_SC("USERDATA
      %p[%p]"),_userdataval(obj),_userdata(obj)->_delegate);break;
02271         case OT_GENERATOR:      scprintf(_SC("GENERATOR %p"),_generator(obj));break;
02272         case OT_THREAD:         scprintf(_SC("THREAD [%p]"),_thread(obj));break;
02273         case OT_USERPOINTER:    scprintf(_SC("USERPOINTER %p"),_userpointer(obj));break;
02274         case OT_CLASS:          scprintf(_SC("CLASS %p"),_class(obj));break;
02275         case OT_INSTANCE:       scprintf(_SC("INSTANCE %p"),_instance(obj));break;
02276         case OT_WEAKREF:        scprintf(_SC("WEAKREF %p"),_weakref(obj));break;
02277         default:
02278             assert(0);
02279             break;
02280         };
02281         scprintf(_SC("\n"));
02282         ++n;
02283     }
02284 }
```

```
02285
02286
02287
02288 #endif
```